BELLMAN–FORD ALGORITHM

1. Definition (Simple & Precise)

Bellman–Ford Algorithm is a Dynamic Programming shortest-path algorithm that finds the minimum distance from a source vertex to all other vertices in a weighted graph, including those with negative edge weights.

It can also detect negative weight cycles.

2. Why Bellman–Ford? (Intuition)

- It repeatedly relaxes every edge.
- After V–1 iterations, all shortest paths are guaranteed to settle.
- A Vth iteration checks if further relaxation is possible
  If yes → Negative cycle exists.

3. When does Bellman–Ford work?

✔ Works on:

- Directed graphs
- Undirected graphs (convert edges to two directions)
- Positive & negative weights
- Graphs containing a risk of negative cycles

❌ Does NOT work on:

- Extremely large graphs (slow)
- Graphs with multiplicative weights (not additive)

4. Why Bellman–Ford Works with Negative Weights?

Because it does not assume greedy correctness.
It checks every edge in every iteration and updates distances until stable.

5. Why Bellman–Ford Detects Negative Cycles?

If after **V–1 relaxations**, any edge can still be improved,
→ A negative cycle is reachable from the source.

6. Key Terms

Term	Meaning
Relaxation	Update if `dist[u] + w < dist[v]`
Iteration	One full pass over all edges
Negative cycle	Sum of cycle weights < 0
Dynamic Programming	Builds answers in rounds

7. Algorithm

```
// Algo BellmanFord(G, source)
// Input: Graph G(V, E) with positive or negative weights
// Output: Shortest dist from source to all nodes

1. Initialize dist[] = ∞ for all nodes
2. dist[source] = 0

3. Repeat (V - 1) times:
       For each edge (u, v, w):
            if dist[u] + w < dist[v]:
                 dist[v] = dist[u] + w
4. // Check for negative cycles
   For each edge (u, v, w):
        if dist[u] + w < dist[v]:
             print "Negative cycle exists"
             stop

5. Return dist[]
```

8. Time Complexity

Implementation	Complexity
Standard	**O(V × E)**
With early stopping	Slightly faster average case

9. Applications

✔ Detecting arbitrage in currency exchange
✔ Routing algorithms in networks
✔ Graphs with negative weights
✔ Longest path in DAG (after weight reversal)
✔ Energy or cost optimization with gains/losses

10. Advantages

✔ Works with negative weights
✔ Detects negative cycles
✔ Simple to implement
✔ Guaranteed correctness for all weights

11. Limitations

- Slower than Dijkstra
- Not suitable for very large graphs
- Cannot give correct result if negative cycle is reachable from source

12. Difference: Bellman–Ford vs Dijkstra

Feature	Dijkstra	Bellman–Ford
Negative edges	❌ No	✔ Yes
Negative cycles	❌ No	✔ Detects
Approach	Greedy	Dynamic Programming

Time complexity	Fast	Slow
Uses	Maps, GPS	Finance, losses, risk graphs